Network Security – Lab 4

Rahul Kumar (252IS029)

Question 1: Analyze the security of the provided JWT implementation and address the identified vulnerabilities.

JSON Web Tokens (JWT)

JWT is a compact token used for securely transmitting information between client and server. It is commonly used for authentication.

Structure of JWT

It consists of three parts:

$$\text{JWT} = \underbrace{\text{Header}}_{\text{Algorithm \& token type}} . \underbrace{\text{Payload}}_{\text{User data (claims)}} . \underbrace{\text{Signature}}_{\text{Ensures integrity}}$$

Working of JWT

User logs in → server creates JWT → JWT is sent to client → Client sends JWT in every request → Server verifies JWT signature → Access is granted if valid.

Algorithm

Encoding JWT

Convert header & payload to Base64URL → Concatenate header + payload → Generate HMAC-SHA256 signature → Form final token.

Verification

Split token into 3 parts → Verify algorithm → Recalculate signature → Compare signatures → Check expiry.

Implementation

```python
import json
import base64
import hmac
import hashlib
import time

# encoding
def base64url_encode(data):
    return base64.urlsafe_b64encode(data).rstrip(b'=')

def base64url_decode(data):
    padding = b'=' * (4 - len(data) % 4)
```

```
    return base64.urlsafe_b64decode(data + padding)

def create_jwt(payload, secret):
    header = {
        "alg": "HS256",
        "typ": "JWT"
    }
    header_b64  = base64url_encode(json.dumps(header).encode())
    payload_b64 = base64url_encode(json.dumps(payload).encode()
       )
    message = header_b64 + b'.' + payload_b64
    signature = hmac.new(
        secret.encode(),
        message,
        hashlib.sha256
    ).digest()
    signature_b64 = base64url_encode(signature)
    token = message + b'.' + signature_b64
    return token.decode()

def verify_jwt(token, secret):
    header_b64, payload_b64, signature_b64 = token.split('.')

    #### none algo part
    header, payload, signature = token.split('.')
    header = json.loads(base64.urlsafe_b64decode(header + "==")
       )

    # soln
    if header["alg"] != "HS256":
        raise Exception("Invalid algorithm")

    if header["alg"] == "none":
        print("Token accepted (no signature check)")
        return json.loads(base64.urlsafe_b64decode(payload + "
           ==")))

    #### end first

    message = (header_b64 + "." + payload_b64).encode()
    expected_signature = hmac.new(
        secret.encode(),
        message,
        hashlib.sha256
    ).digest()
    expected_signature_b64 = base64url_encode(
       expected_signature)

    if expected_signature_b64.decode() != signature_b64:
        return "Invalid Token"
```

```
    payload = json.loads(base64url_decode(payload_b64.encode())
       )

    # checking expiration of token
    if "exp" in payload and payload["exp"] < int(time.time())
       and payload["exp"] > 20:
        return "Token Expired"

    return payload



secret  = "iHaveToDoITAnyHow"
payload = {
    "user_id": 1,
    "role": "user",
    "exp": int(time.time()) + 3600
}

token = create_jwt(payload, secret)
print("\nTOKEN:", token)
print("\nVERIFY:", verify_jwt(token, secret))

header1 = {"alg": "none", "typ": "JWT"}
payload1 = {"user_id": 1, "role": "admin"}

header_b64  = base64url_encode(json.dumps(header1).encode())
payload_b64 = base64url_encode(json.dumps(payload1).encode())
token1 = header_b64 + b'.' + payload_b64 + b'.'
print("\n modified token ", token1.decode())

tok = token1.decode()









## 2nd Attack
token_bruteforce = token
header_b64, payload_b64, signature_b64 = token_bruteforce.split
   ('.')
message = (header_b64 + "." + payload_b64).encode()

secret_bruteforce = ""
with open("wordlist.txt", "r") as file:
    for line in file:
        secret_bruteforce = line.strip()
        signature = hmac.new(
```

```
            secret_bruteforce.encode(),
            message,
            hashlib.sha256
        ).digest()
        signature_test = base64url_encode(signature)
        print("Trying:", secret_bruteforce)
        if signature_test.decode() == signature_b64:
            print("Secret Found:", secret_bruteforce)
            break

payload = {
    "user_id": 1,
    "role": "admin",
    "exp": int(time.time()) + 3
}

token3rd = create_jwt(payload, secret_bruteforce)  # using
   bruteforce secret key
print("\nTOKEN:", token3rd)
print("\nVerification output:", verify_jwt(token3rd,
   secret_bruteforce))





## 3rd when token was stolen and used for replay
# if HTTPS was not used "Wireshark, MITM attack"
time.sleep(1)
print("\nREplay\nVerification output:", verify_jwt(token3rd,
   secret_bruteforce))



# token expire
time.sleep(1)
print("\nToken not Expired\nVerification output:", verify_jwt(
   token3rd, secret_bruteforce))
time.sleep(5)
print("\nTK expired\nVerification output:", verify_jwt(token3rd
   , secret_bruteforce))
```

Vulnerability Analysis and Mitigation

1. None Algorithm Attack

Vulnerability: Attackers can modify the JWT header to set the algorithm to `"none,"` effectively stripping the signature requirement and allowing them to forge claims.

alg: none — Some servers trust the header without validation.

Instead of this:

```
if header["alg"] == "none":
    print("Token accepted (no signature check)")
```

We have to check:

```
if header["alg"] != "HS256":
    raise Exception("Invalid algorithm")

if header["alg"] == "none":
    print("Token accepted (no signature check)")
```

2. Secret Key Brute Force

Vulnerability: If a weak, short, or predictable secret key is used, attackers can use offline brute-force tools to discover the key and sign their own tokens.

Attacker tries multiple keys to guess the secret:

```
with open("wordlist.txt", "r") as file:
    for line in file:
        ...
```

Weak secrets like `"password123"` can be guessed.

3. Token Replay

Vulnerability: A valid token intercepted by an attacker can be reused (replayed) until it expires, gaining unauthorized access.

Stolen token is reused by attacker:

```
# if HTTPS was not used "Wireshark, MITM attack"
time.sleep(1)
print("REplay\nVerification output:", verify_jwt(token3rd,
   secret_bruteforce))
```

4. Expiry Not Checked

Vulnerability: Tokens without an expiration time remain valid indefinitely, increasing the window of opportunity for attackers if a token is compromised.

If expiry is ignored, the token becomes permanent.

Prevention: Always validate the `exp` claim:

```
# checking expiration of token
if "exp" in payload and payload["exp"] < int(time.time()) and
   payload["exp"] > 20:
    return "Token Expired"
```

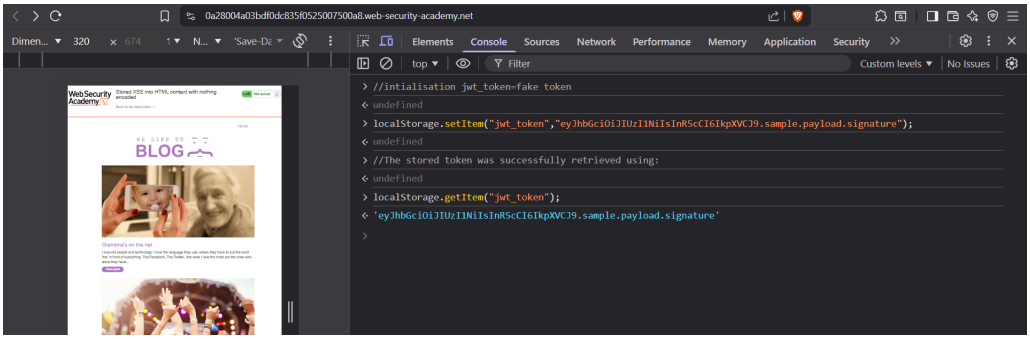
5. Storing in Local Storage

JWT tokens stored in the browser's `localStorage` can be stolen through XSS attacks. An attacker can inject malicious JavaScript into a website (XSS vulnerable):

```
<script>
  console.log(localStorage.getItem("jwt_token"))
</script>
```

which can read the token and send it to the attacker's server.

Example:




```
// initialisation: jwt_token = fake token
localStorage.setItem("jwt_token",
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.sample.payload.
     signature");

// The stored token was successfully retrieved using:
localStorage.getItem("jwt_token");
```

Mitigation: JWT tokens should be stored in HTTP-only cookies instead of `localStorage`.

Question 2: Implement and analyze the WebAuthn protocol and compare the performance costs of symmetric vs asymmetric authentication.

WebAuthn

WebAuthn is an authentication standard that uses public-key cryptography instead of passwords. It allows secure login using devices like biometrics or hardware keys.

Working:

1. **Registration Phase**
   User generates key pair.
   Public key → stored on server.
   Private key → stays with user.

2. **Authentication Phase**
   Server generates random challenge → User signs challenge using private key → Server verifies using public key → If valid → access granted.

Server stores public key; User device stores private key; Authentication is done by signing a challenge.

Algorithm – WebAuthn Flow

Generate RSA key pair → Store public key in server DB → Generate challenge → Sign challenge → Verify signature.

Implementation

```
import os

# Registration (generate key pair)
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

def register():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
    public_key = private_key.public_key()
    return private_key, public_key

# SERVER Database
server_db = {}
private_key, public_key = register()
print("\nPrivate Key Generated :", private_key)
print("\nPublic Key Generated :", public_key)
server_db["user1"] = public_key  # server stores public key
```

```python
# fun to create random challenge by server and send to
    authenticator/user
def create_challenge():
    return os.urandom(32)


# Sign Challenge (Authenticator)
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding

def sign_challenge(private_key, challenge):
    signature = private_key.sign(
        challenge,
        padding.PKCS1v15(),
        hashes.SHA256()
    )
    return signature


# VERIFY Signature (Server)
def verify_signature(public_key, challenge, signature):
    try:
        public_key.verify(
            signature,
            challenge,
            padding.PKCS1v15(),
            hashes.SHA256()
        )
        print("Authentication Success")
    except Exception:
        print("Authentication Failed")


# FULL FLOW
challenge = create_challenge()
print("\n\nChallenge (hex):")
print(challenge.hex(), "\n")

print("\n==> Authenticator signs the challenge using PRIVATE
    KEY")
signature = sign_challenge(private_key, challenge)
print("\nSignature Created")
print("\nSignature (first 64 hex chars):")
print(signature.hex()[:64], "...", "\n")

print("\n==> now verification ")
verify_signature(server_db["user1"], challenge, signature)
```

Comparison

Asymmetric Time → WebAuthn

```
# comparison
print("\n\n==> Now Comparison :")
import time

start = time.time()
signature = sign_challenge(private_key, challenge)
verify_signature(server_db["user1"], challenge, signature)
print("Asymmetric time:", (time.time() - start) * 1000000)
```

Output:

```
==> Now Comparison :
VERIFY: {'user_id': 1, 'role': 'admin', 'exp': 1773823521}
Symmetric time: 186.2049102783203
(in micro sec)
```

Symmetric Time → JWT

```
# comparison
print("\n\n==> Now Comparison :")
import time

payload = {
    "user_id": 1,
    "role": "admin",
    "exp": int(time.time()) + 20
}

start = time.time()
token = create_jwt(payload, secret)
print("\nVERIFY:", verify_jwt(token, secret))
print("Symmetric time:", (time.time() - start) * 1000000)
```

Output:

```
==> Now Comparison :
Authentication Success
Asymmetric time: 258.9225769042969
```

	Symmetric	**Asymmetric**
Key Type	Single shared key	Public + private key
Speed	Fast	Slower
Security	Less secure	Highly secure
Key distribution	Difficult	Easy